

林凯-264db0022-项目申请书

项目申请书

项目	内容
项目编号	264db0022
项目名称	让 AgentSight 扛住生产级流量：eBPF 采集管线的内存与稳定性攻坚
项目导师	舒义
申请人	林凯
日期	2026.8.24
邮箱	peanut_l@qq.com

一、项目背景

1. 项目描述

AgentSight 是一套面向 AI Agent 的可观测工具，不改一行 Agent 代码，用 eBPF 在内核层零侵入地抓取 LLM API 调用、token 消耗与进程行为。数据流经 Probes → Parser → Aggregator → Analyzer → GenAI → Storage 管线，完成 HTTP/HTTP2/SSE 解析、请求—响应关联与 token 计量，最终落存储并可视化，目前已能端到端跑通一条完整 trace。

但在高流量、长时间运行下，AgentSight 存在内存开销偏高与稳定性不足的问题：

- 缺乏背压**：采集入口在下游积压时缺少背压，事件不断进入，内存随负载增长；
- 连接缓存约束单一**：连接缓存仅按数量约束，缺少字节量与存活时间维度的控制，在途连接可能被挤出而丢失关联；
- 热路径易 panic**：热路径遇到异常输入时容易 panic，长时间运行存在资源不能及时释放的隐患；
- 缺少量化基线**：缺少可复现的压测基线与 before/after 对比，稳定性改进难以量化验证。

本项目希望在不牺牲采集完整性的前提下，让采集管线在持续高负载下内存有明确上界、面对异常输入能容错降级不崩溃，并建立可复现的压测基线。最终目标是让 AgentSight 采集管线达到生产可用：持续高负载下内存有稳定上界、不随运行时间线性增长，全程无 OOM 与 panic 退出、采集完整性不后退，配套 before/after 量化报告。

2. 项目需求

根据项目要求，最终需要交付以下内容：

1. **基线与压测**：可复现的负载生成与基准脚本；内存/吞吐/延迟/丢弃率的 before/after 量化报告；
2. **内存治理**：让管线在持续高负载下内存有上界的实现（含设计说明与取舍依据）；
3. **异常输入容错与资源泄漏治理的实现**：对应的回归测试。

在工程质量上，需要遵循仓库硬性规则：提交前运行 `cargo fmt + cargo clippy (-D warnings) + cargo test`；非测试代码禁止 `unwrap/expect`；PR 不超过 800 行，复杂逻辑不超过 500 行；遵循 L0-L8 分层约束；流水线变更必须包含集成测试。

3. 项目地址

<https://github.com/alibaba/anolisa/tree/main/src/agentsight>

二、对项目与源码方向的理解

注：本节基于我对 AgentSight 管线架构、配置体系与 eBPF 采集链路的理解进行设计说明。正式开发前，我会进一步阅读相关模块源码，并与导师确认最终落点。

1. AgentSight 管线架构理解

AgentSight 的数据流水线为 Probes → Parser → Aggregator → Analyzer → GenAI → Storage：

1. **Probes (src/probes/)**：eBPF 探针管理，包括 `sslsniff`（SSL_read/SSL_write 捕获加密流量明文）、`proctrace`（`execve` 捕获命令行参数）、`procmon`（进程创建/退出，Agent 发现）、`filewatch/filewrite`（`.jsonl` 文件监控）、`udpdns`（DNS 查询）、`TcpSniff` 等；
2. **Event (src/event.rs)**：统一事件枚举（`Ssl/Proc/ProcMon/FileWatch/FileWrite/UdpDns`）；
3. **Parser (src/parser/)**：协议解析，支持 HTTP/1.x、HTTP/2、SSE 与 ProcTrace；
4. **Aggregator (src/aggregator/)**：请求-响应关联 + SSE continuation buffer（如 `HttpConnectionAggregator` 在 `SseActive` 状态下为 `/v1/responses` 路径缓冲原始 SSL 字节）；
5. **Analyzer (src/analyzer/)**：Token/审计/消息分析；
6. **GenAI (src/genai/)**：语义事件构建与导出（`GenAIBuilder`、`GenAISemanticEvent`、`GenAIExporter`）；
7. **Storage (src/storage/)**：SQLite 持久化（`SqliteStore`、`AuditStore`、`TokenStore`），支持 `features` 开关（关闭时用 `noop store` 减少内存与 I/O 开销）。

此外还包括 `Discovery`（`AgentScanner/AgentMatcher`，Agent 自动发现）、`Health`、`Tokenizer`（`LlmTokenizer/MultiModelTokenizer`）、`ATIF` 轨迹导出、`Server`（HTTP API + 嵌入式前端）、`Container`（容器 ID 提取）、`Config`（`AgentsightConfig`）与 `Unified`（AgentSight 主编排器）。

2. 配置与内存控制现状理解

AgentSight 已具备一定的资源控制能力：

1. **runtime_limits**：配置有界缓冲区上限，防止内存无限增长；
2. **event_channel_policy**：满载策略支持 backpressure / drop_newest / sample；
3. **features 开关**：sqlite_storage 等可独立启停，关闭后对应模块不实例化（Storage::noop 等），减少内存与 I/O 开销。

这意味着"内存上界"的实现已有部分基础设施（有界缓冲区、满载策略、特性开关），本项目需要在此基础上补齐系统性治理：

1. 采集入口的背压/降级策略要覆盖全链路（而不只是事件通道）；
2. 连接缓存要从"仅数量约束"升级为"数量 + 字节量 + TTL"多维约束；
3. 热路径异常输入的容错（消除 panic 路径）与资源泄漏治理需要系统化；
4. 全部改进需要可复现压测基线与 before/after 量化报告。

3. 本项目需要解决的核心问题

1. **内存无界增长**：下游积压时入口仍接收事件，缓冲与在途连接导致内存随负载/时间线性增长；
2. **连接关联丢失**：连接缓存仅按数量约束，字节量大或存活久的连接被挤出，请求-响应关联丢失；
3. **热路径崩溃**：异常输入导致 panic，整条管线退出；
4. **资源泄漏**：长时间运行下 fd、buffer、连接等资源不能及时释放；
5. **缺少量化验证**：无统一的压测基线与稳定性指标，无法证明改进效果。

三、项目需求分析

1. 基线与压测需求

1.1 负载生成

需要可复现的负载生成器，覆盖：

1. **正常负载**：稳定的 LLM API 调用流（HTTP/SSE）；
2. **峰值负载**：突发高并发请求；
3. **异常输入**：畸形 HTTP/HTTP2/SSE 数据、超长头部、截断帧、非法 UTF-8；
4. **长时运行**：持续数小时以上的运行验证内存上界。

1.2 指标采集

基线指标包括：

- 内存：常驻内存（RSS）、堆内存、缓冲区占用、连接缓存字节量；
- 吞吐：事件处理速率、解析速率、存储速率；

- 延迟：P50/P95/P99 事件处理延迟；
- 丢弃率：事件丢弃数/采样数；
- 稳定性：OOM 次数、panic 次数、进程重启次数。

1.3 before/after 报告

同一环境、同一负载模型下，改进前后的量化对比报告。

2. 内存治理需求

2.1 背压与降级

1. **入口背压**：当下游积压达到阈值时，采集入口按 event_channel_policy 执行 backpressure / drop_newest / sample；
2. **全链路有界**：Probes→Parser→Aggregator→Storage 每个环节的队列与缓冲都有上界；
3. **取舍明确**：每种策略的取舍（完整性 vs 内存 vs 延迟）要有文档说明与配置；
4. **指标可见**：背压触发次数、丢弃数、队列水位可观测。

2.2 连接缓存治理

1. **多维约束**：连接缓存从"仅数量"升级为"数量 + 字节量 + TTL"；
2. **淘汰策略**：按字节量/存活时间淘汰，避免在途连接被随意挤出；
3. **关联保护**：对活跃关联（如 SSE continuation）的连接做保护，防止关联丢失；
4. **配置化**：阈值可配置，默认值有依据。

2.3 缓冲治理

1. 审计所有无界容器（Vec/HashMap/队列），补上界；
2. SSE continuation buffer、解析中间缓冲等按字节量限制；
3. 缓冲溢出时降级（截断/丢弃并计数），不 panic。

3. 容错与资源治理需求

3.1 异常输入容错

1. 解析器对畸形输入返回错误而非 panic；
2. 热路径（SSL 事件解析、HTTP/SSE 解析、token 提取）全部无 panic 路径；
3. 单条异常数据不导致整条管线中断；
4. 错误计数与日志，便于问题定位。

3.2 资源泄漏治理

1. 审计 fd、buffer、连接、线程的创建与释放；
2. 修复长运行下的资源泄漏点；
3. 提供资源使用监控（fd 数、连接数、buffer 池占用）；
4. 长时运行回归测试兜底。

4. 工程质量需求

遵循仓库硬性规则：cargo fmt + cargo clippy -D warnings + cargo test；禁止 unwrap/expect（非测试代码）；PR ≤800 行；L0-L8 分层约束；流水线变更含集成测试；FFI 变更同步 cbindgen.toml 与 drift guard。

四、项目实施方案

1. 整体架构

我计划按"基线 → 治理 → 容错 → 验证"四个阶段组织实现，改动遵循"扩展现有模块优先"的 Footprint Ladder 原则：

1. **基线阶段**：搭建负载生成与压测框架，采集 before 数据；
2. **治理阶段**：实现入口背压/降级与连接缓存多维约束；
3. **容错阶段**：消除热路径 panic，治理资源泄漏；
4. **验证阶段**：长时压测验证内存上界，输出 after 报告与回归测试。

2. 模块划分

2.1 压测与基线模块

- `LoadGenerator`：可复现负载生成（正常/峰值/异常输入/长时）；
- `MetricsCollector`：内存/吞吐/延迟/丢弃率指标采集；
- `BaselineReporter`：before/after 报告生成；
- `StressHarness`：压测编排（场景、时长、环境记录）。

2.2 背压与队列治理模块

- `BackpressureController`：入口背压状态机（正常/警告/饱和/恢复）；
- `BoundedChannel`：全链路有界队列（容量 + 水位 + 溢出策略）；
- `OverflowPolicy`：backpressure / drop_newest / sample 策略实现；
- `WatermarkMetrics`：队列水位、背压次数、丢弃数指标。

2.3 连接缓存治理模块

- `ConnectionCache`：连接缓存（数量 + 字节量 + TTL 多维约束）；

- `EvictionPolicy`：按字节量/存活时间/活跃度淘汰；
- `AssociationGuard`：活跃关联（SSE continuation）保护；
- `CacheMetrics`：缓存占用、淘汰数、关联丢失数指标。

2.4 容错模块

- `ParserHardening`：HTTP/HTTP2/SSE/ProcTrace 解析器异常输入容错；
- `PanicAudit`：热路径 panic 审计与消除；
- `ErrorRecovery`：单条异常数据的隔离与降级；
- `ResourceAudit`：fd/连接/buffer/线程泄漏审计与修复；
- `ResourceMonitor`：资源使用监控与告警。

2.5 验证模块

- `RegressionTests`：异常输入与资源泄漏回归测试；
- `LongRunTest`：长时运行内存上界测试；
- `IntegrationTests`：管线级集成测试
(probes→parser→aggregator→analyzer→genai→storage)。

3. 数据结构设计

3.1 连接缓存条目

代码块

```
1  pub struct ConnectionEntry {
2      pub connection_id: ConnectionId,
3      pub bytes_buffered: usize,
4      pub last_active_at: Instant,
5      pub created_at: Instant,
6      pub association: AssociationState, // Idle / ActiveSse / AwaitingResponse
7      pub parser_state: ParserState,
8  }
```

3.2 有界队列

代码块

```
1  pub struct BoundedQueue<T> {
2      pub capacity: usize,
3      pub watermark: usize,
4      pub policy: OverflowPolicy,
5      pub dropped: AtomicU64,
```

```
6     pub  enqueue: AtomicU64,  
7     queue: VecDeque<T>,  
8 }
```

3.3 背压状态

代码块

```
1  pub  enum  BackpressureState {  
2      Normal,  
3      Warning { queue_depth:  usize },  
4      Saturated { policy: OverflowPolicy },  
5      Recovering,  
6  }
```

4. 接口设计

4.1 配置扩展 (agentsight.json)

代码块

```
1  {  
2      "runtime_limits": {  
3          "connection_cache": {  
4              "max_connections": 10000,  
5              "max_total_bytes": 67108864,  
6              "max_idle_ttl_secs": 300,  
7              "protect_active_associations": true  
8          },  
9          "queue": {  
10             "event_channel_capacity": 100000,  
11             "parser_channel_capacity": 50000,  
12             "watermark_ratio": 0.8  
13         }  
14     },  
15     "event_channel_policy": "backpressure"  
16 }
```

4.2 指标接口

代码块

```
1  pub  struct  StabilityMetrics {  
2      pub  memory_rss_bytes: u64,  
3      pub  queue_watermarks: BTreeMap<String,  usize>,  
4  }
```

```
4     pub dropped_events: u64,  
5     pub backpressure_events: u64,  
6     pub cache_bytes: u64,  
7     pub evicted_connections: u64,  
8     pub lost_associations: u64,  
9     pub panics: u64,  
10    pub open_fds: u64,  
11 }
```

五、开发计划

本项目开发周期为 3 个月。我计划将开发过程分为四个阶段。

第一阶段：基线问题定位（第 1-3 周）

主要目标：建立可复现压测基线，定位内存热点与 panic 路径。

具体工作：

1. 阅读 AgentSight 管线代码（probes/parser/aggregator/analyzer/genai/storage）；
2. 梳理所有无界缓冲、连接缓存与热路径 panic 点；
3. 实现负载生成器与指标采集；
4. 运行正常/峰值/异常输入/长时场景，输出 before 基线报告；
5. 与导师确认治理优先级。

阶段产出：

- 压测框架与负载生成器；
- before 基线报告；
- 问题清单与治理方案。

第二阶段：内存治理（第 4-8 周）

主要目标：实现背压/降级与连接缓存多维约束。

具体工作：

1. 实现入口背压状态机与全链路有界队列；
2. 实现 OverflowPolicy（backpressure/drop_newest/sample）；
3. 实现连接缓存的数量 + 字节量 + TTL 约束与淘汰策略；
4. 实现活跃关联保护；
5. 补充队列/缓存单元测试与水位指标；
6. 中期压测验证内存上界。

阶段产出：

- 背压与队列治理；
- 连接缓存治理；
- 中期量化结果。

第三阶段：容错与资源治理（第 9-11 周）

主要目标：消除热路径 panic，治理资源泄漏。

具体工作：

1. 审计并修复解析器热路径 panic（HTTP/HTTP2/SSE/token 提取）；
2. 实现单条异常数据隔离与降级；
3. 审计并修复 fd/连接/buffer/线程泄漏；
4. 实现资源使用监控；
5. 补充异常输入与资源泄漏回归测试。

阶段产出：

- 容错与资源治理实现；
- 回归测试；
- 中期稳定性验证。

第四阶段：验证、报告与合入（第 12 周）

主要目标：长时压测验证、before/after 报告与代码合入。

具体工作：

1. 运行长时压测（正常 + 峰值 + 异常输入混合）；
2. 验证内存稳定上界、无 OOM/panic、采集完整性不回退；
3. 输出 before/after 量化报告与设计取舍说明；
4. 提交代码 PR（遵循 ≤ 800 行、L0-L8、无 unwrap 等规则）；
5. 完善文档。

阶段产出：

- before/after 量化报告；
- 合入 PR；
- 使用与设计文档。

六、测试方案

1. 单元测试

1. BoundedQueue 容量/水位/溢出策略测试；
2. ConnectionCache 多维约束与淘汰测试；
3. 背压状态机转换测试；
4. 解析器异常输入容错测试（畸形 HTTP/SSE/HTTP2）；
5. 资源释放测试。

2. 集成测试

1. 管线级集成测试（probes→parser→aggregator→analyzer→genai→storage）；
2. 异常输入注入 → 管线继续运行测试；
3. 连接缓存溢出 → 关联丢失计数测试；
4. 长时运行内存上界测试；
5. 特性开关（noop store）内存收益测试。

3. 压测验收

1. 负载模型可复现；
2. 高负载下内存有稳定上界、不随运行时间线性增长；
3. 全程无 OOM 与 panic 退出、采集完整性不回退；
4. before/after 报告包含内存/吞吐/延迟/丢弃率。

七、风险分析与应对方案

1. 背压导致采集完整性下降

风险：背压/丢弃策略可能降低采集完整性，与"完整性不回退"目标冲突。

应对：优先 backpressure（让消费者跟上），饱和时再降级；量化丢弃率并纳入报告；策略可配置并说明取舍。

2. 连接缓存约束破坏关联

风险：字节量/TTL 约束可能挤掉活跃连接，导致请求-响应关联丢失。

应对：活跃关联保护（SSE continuation 等）；淘汰顺序按活跃度；关联丢失计数可见。

3. 热路径改动引入回归

风险：消除 panic 与新增容错可能改变解析行为。

应对：解析行为用既有测试锁定；新增容错不改变正常路径语义；集成测试覆盖管线级行为。

4. 压测环境不可复现

风险：内核版本、eBPF 探针行为、负载模型差异导致结果不可比。

应对：固定环境信息（内核、硬件、配置）并记录；负载模型脚本化；before/after 同一环境运行。

5. 资源泄漏定位困难

风险：长时泄漏点隐蔽，复现成本高。

应对：资源监控（fd/连接/buffer）+ 长时压测 + 逐步二分定位；修复后保留回归测试。

八、个人能力与相关经历

我是福州大学软件工程专业本科在读学生，技术栈以 Java 后端与分布式系统为主，对性能、内存与稳定性治理有实际工程经验。

在工程能力方面，我独立完成过一个基于 SpringBoot3 + WebSocket + RabbitMQ + Redis + MySQL 的分布式即时通信系统，涉及连接管理、消息队列、缓存、限流与可靠投递。这段经历让我对以下与本项目高度相关的能力有实际理解：

- **背压与有界队列**：在实践中处理过消息积压、消费能力与限流，理解"有界队列 + 水位 + 溢出策略"的背压思想；
- **连接与缓存治理**：维护过多端登录连接、未读计数缓存与离线同步，理解连接生命周期、TTL 与淘汰策略；
- **资源与稳定性**：处理过连接释放、超时与重试，理解 fd/缓冲/线程资源治理与优雅关闭；
- **性能方法论**：熟悉"先基线、再剖析、后优化、用数据验证"的方法，能够输出量化报告；
- **Rust 学习基础**：已开始系统学习 Rust 的所有权/生命周期与内存行为，本项目的内存上界与容错工作正是 Rust 所有权模型擅长的场景，并将严格遵循仓库的 no-unwrap/clippy/PR 规模规则；
- **AI Agent 可观测**：关注 Agent 的多步决策与轨迹概念，理解"不侵入 Agent 代码做可观测"的价值。

我申请本项目的原因主要有三点：

1. 本项目是"内存上界 + 容错降级 + 量化验证"的系统性稳定性工程，与我的性能治理兴趣高度契合；
2. eBPF 可观测是 AI Agent 基础设施的重要方向，项目技术含量高；
3. AgentSight 有完善的架构文档与硬性质量规则，适合系统学习 Rust 工程实践。

九、预期成果

1. **基线与压测**：可复现负载生成器、压测框架与 before/after 量化报告；
2. **内存治理**：入口背压/降级、全链路有界队列、连接缓存多维约束与活跃关联保护；
3. **容错与资源治理**：热路径 panic 消除、单条异常隔离、资源泄漏修复与监控；
4. **验证**：长时压测证明内存上界、无 OOM/panic、完整性不回退；

5. 合入与文档：遵循仓库规则合入的 PR 与设计/使用文档。

十、总结

本项目的核心目标是把 AgentSight 从"能跑通"升级为"生产可用"：在持续高负载下内存有明确上界、面对异常输入不崩溃、长时间运行不泄漏，并且所有改进都有可复现的量化证据。

我计划以"基线 → 治理 → 容错 → 验证"为主线，复用现有

runtime_limits/event_channel_policy/features 基础设施，遵循仓库的工程质量规则，最终交付内存、稳定与完整性兼顾的采集管线以及完整的 before/after 报告。